

Heterogeneous ML·DL Ensemble for Reliable Lithography Hotspot Detection on the ICCAD-12 Benchmarks

****Subham Prasad Gupta**** (Reg. No. 23BVD1039)

****Akhil Kumar Kudipudi**** (Reg. No. 23BVD1042)

****Tejasvini R**** (Reg. No. 23BVD1044)

B.Tech. (BVD), VIT Chennai, India

Group 6 · Digital Assignment II · BEVD402L: AI and Machine Learning for IC · Faculty: Dr. G. Lakshmi Priya

Abstract

Lithography hotspots are layout patterns that print with open or short defects, and they must be found early in the design flow because rigorous lithography simulation is too slow for full-chip screening. A lightweight CNN trained from scratch was recently shown to beat VGG16 transfer-learning pipelines on the ICCAD-12 benchmarks. This work asks whether combining different learning algorithms makes hotspot detection more reliable than that single CNN. We build ensembles of the CNN's own sigmoid head and an SVM, a Random Forest and an ANN trained on the CNN's 64-D embedding, using majority voting, weighted voting and stacking, and compare them with the CNN baseline on all five ICCAD-12 benchmarks under identical conditions, reporting balanced accuracy (BA), precision, recall, specificity, F1 and confusion matrices. Stacking is the best configuration: average BA rises from 92.09% to 92.97%, and the number of missed hotspots pooled over the five test sets falls from 234 to 148 (·37%) at an unchanged false-alarm count (3,502 vs 3,500). Majority and weighted voting do not improve BA (89.92% and 90.03%) but raise precision and F1 at the cost of recall. A leave-one-out ablation shows the CNN's own output is the most valuable member, whereas the ANN, which collapses to predicting only non-hotspots on ICCAD-5, slightly hurts the weighted-vote ensemble. The gains are not uniform: stacking trails the baseline on the two most imbalanced benchmarks, and the average improvement is of the same order as the sampling uncertainty of a single run. Because all members share one embedding, their errors are correlated.

****Index Terms****.lithography hotspot detection, ICCAD-12, ensemble learning, stacking, class imbalance, balanced accuracy, convolutional neural network.

I. INTRODUCTION

Photolithography transfers a circuit layout onto silicon, but as feature sizes shrink relative to the exposure wavelength, diffraction distorts the printed pattern. Layout patterns that print with pinching (open-circuit) or bridging (short-circuit) defects are called lithography hotspots [1]. Rigorous lithography simulation finds them accurately but is far too slow for full-chip screening, which motivated fast pattern matching [2] and machine-learning detectors [3], [4], [5], [6], [7]. The ICCAD-2012 contest suite [8] has become the standard testbed. Its five benchmarks are extremely imbalanced·hotspots are rare·so plain accuracy is misleading and balanced accuracy (BA) together with confusion-matrix measures must be used [9].

The reference work supplied for this assignment [10] compares two routes on ICCAD-12: (i) features from a pre-trained VGG16 [11] fed to classical classifiers [12], [1], whose best variant (a linear SVM) reaches 80.94% average BA at 182 ms per clip, and (ii) a 12,873-parameter CNN trained from scratch [13] that reaches 95.3% average validation BA at 6.3 ms per clip. It concludes that the lightweight CNN is preferable. However, the models are treated only as competitors, they are never combined, and results are shown mainly as BA bar charts, without per-benchmark confusion matrices, precision or F1, so the trade-off between missed hotspots and false alarms cannot be judged.

****Research gap.**** The classifier heads that can sit on top of CNN features·a sigmoid layer, an SVM, a Random Forest, a multilayer perceptron·have different decision rules and may therefore make different mistakes, but combining them has not been explored on ICCAD-12. ****Research question.**** Can combining predictions from heterogeneous ML/DL models (the CNN head and an SVM, Random Forest and ANN built on the CNN's

embedding) improve hotspot-detection reliability compared with the individual CNN baseline, and which error patterns does the combination reduce? ·Heterogeneous· here refers to the learning algorithm, not to the input: all members share one learned representation. We test two hypotheses: **H1**·a learned combination (stacking) achieves higher average BA than the best single model; **H2**·the ensemble reduces missed hotspots (false negatives) without a proportional rise in false alarms. The contributions are:

- three combination strategies (majority vote, weighted vote, stacking) over four members (the CNN head and an SVM, Random Forest and ANN built on the same CNN embedding), compared with the common baseline on all five benchmarks under identical conditions;
- per-benchmark results with full confusion matrices and an analysis of which error type (false negative or false positive) each combination reduces;
- a leave-one-out ablation quantifying each member's contribution; and
- an explicit account of where the ensemble does *not* help and of the statistical uncertainty of the comparison.

II. BACKGROUND AND RELATED WORK

A. Hotspot Detection

Early detectors rely on pattern matching against a library of known hotspots, which cannot flag unseen patterns [2]. Machine-learning methods replace the library with learned classifiers over engineered features, e.g., topological classification with critical-feature extraction [3], AdaBoost with simplified features [4], and information-theoretic feature optimisation with online learning [5]. Deep learning moves feature extraction into the network: imbalance-aware CNN training [6], feature-tensor inputs with biased learning [7], and comparisons from shallow to deep models [1]. Transfer learning from ImageNet-pretrained VGG16 [11] was studied in [12], whereas [13] showed that a very small CNN trained from scratch is sufficient; this CNN is the common baseline of [10] and of this work.

B. Ensemble Learning

An ensemble improves reliability when its members are individually competent and make diverse errors [14]; members that share one feature representation are less diverse than independently trained ones, which limits what combination can achieve. Voting fixes the combination rule in advance, whereas stacked generalisation trains a meta-learner on the members' held-out outputs so that it learns how far to trust each member [15]; a survey is given in [16]. Because all members here are trained on the same imbalanced data, we evaluate every combination with BA rather than accuracy.

III. DATASET AND CLASS-IMBALANCE ANALYSIS

A. ICCAD-12 Statistics

ICCAD-12 provides five benchmarks, each with a training and a test partition of hotspot (HS) and non-hotspot (NHS) layout clips [8]. Table I lists the counts. The benchmarks are treated separately throughout, as required, and are never merged. Imbalance grows from ICCAD-1 to ICCAD-5: the NHS:HS ratio in the training data rises from 3.4:1 to 104.5:1, and in every benchmark the test set is more imbalanced than the training set (20.7:1 to 471.4:1). ICCAD-5 has only 26 training and 41 test hotspots, so a single hotspot moves its recall by 2.4 percentage points (pp).

TABLE I: ICCAD-12 CLASS DISTRIBUTION

Bench.	Train HS	Train NHS	Ratio	Test HS	Test NHS	Ratio
---	---	---	---	---	---	---
B1	99	340	3.4:1	226	4,679*	20.7:1
B2	174	5,285	30.4:1	498	41,298	82.9:1
B3	909	4,643	5.1:1	1,808	46,333	25.6:1
B4	95	4,452	46.9:1	177	31,890	180.2:1
B5	26	2,716	104.5:1	41	19,327	471.4:1

*Training counts as listed in [10]; test counts are TP+FN and FP+TN of our confusion matrices. *[10] lists 3,869 NHS test clips for ICCAD-1, whereas our pipeline yields 4,679; all ICCAD-1 metrics here use our own counts.*

B. Evaluation Metrics and Why Accuracy Fails

With TP, FP, FN and TN taken from the confusion matrix (HS is the positive class), we use precision $P = \frac{\text{TP}}{\text{TP} + \text{FP}}$, recall (sensitivity) $R = \frac{\text{TP}}{\text{TP} + \text{FN}}$, specificity $S = \frac{\text{TN}}{\text{TN} + \text{FP}}$, $F_1 = 2PR/(P+R)$ and the primary metric:

$$BA = \frac{\text{Sensitivity} + \text{Specificity}}{2} \quad \text{tag1}$$

Accuracy is uninformative here. On ICCAD-5 the ANN member predicts every clip as NHS and still obtains 99.79% accuracy, but its BA is exactly 50%. Precision is also intrinsically low for any detector on such data: with hotspot prevalence π ,

$$P = \frac{\pi R}{\pi R + (1 - \pi)(1 - S)} \quad \text{tag2}$$

so for ICCAD-5 ($\pi = 0.21\%$) even $R = 95.1\%$ and $S = 98.5\%$ yield only $P = 11.7\%$. We therefore interpret precision and F1 relative to prevalence and rank models primarily by BA, recall and specificity.

IV. PROPOSED METHODOLOGY

A. Baseline: Lightweight CNN

The baseline is a lightweight CNN in the style of [13] as adopted in [10]. Each ·basic block· stacks three 3×3 convolutions (ELU, ELU, linear activation), batch normalisation, ELU and 2×2 max-pooling. We use two blocks with 16 and 32 filters, followed by flatten, dropout (0.3), a 64-unit ReLU layer that serves as the embedding, dropout (0.3) and a sigmoid output unit. This network has about 323 k parameters, roughly 25 times the 12,873 of the network in [10], which uses 12 filters per block; the baseline is therefore not a reproduction of the reference architecture, and absolute numbers are not directly comparable with [10]. Training uses Nadam (learning rate 10^{-3}), binary cross-entropy with the hotspot class weighted by the NHS:HS ratio of the training split, batch size 32, at most 15 epochs and early stopping on validation AUC. The same trained CNN is also the first ensemble member, so the baseline-versus-ensemble comparison isolates the effect of combination (the two rows are numerically identical in our results).

B. Ensemble Members (Decision Heads)

Four decision heads form the ensemble (Fig. 1): the CNN’s own sigmoid head (trained end-to-end), and three classical classifiers trained on the frozen 64-D CNN embedding of the training split: an SVM with RBF kernel, a Random Forest and an ANN (multilayer perceptron). Each member m outputs a hotspot probability $p_m(x)$ for a layout clip x . The heads differ in their decision rule (sigmoid boundary, maximum margin, bagged trees, dense network) but share one representation, so their errors are correlated; the ensemble is heterogeneous in learning algorithm rather than in input features.

ICCAD-12 layout clip (grayscale, 48x48, scaled to [0, 1])

·
·

CNN backbone (2 basic blocks, trained end-to-end)

· 64-D embedding

·

.....

· · ·

· · ·

.....

· CNN head · · SVM · · RF · ... (ANN)

· (sigmoid) · · (RBF) · · (MLP) ·

.....

· $p_m(x)$ · $p_m(x)$ · $p_m(x)$

.....

.
 .

 . . .
 . . .
 Majority Vote Weighted Vote Stacking Meta-Learner
 . . .

 .
 .

Decision: hotspot (HS) / non-hotspot (NHS) · common evaluation
 ...

Fig. 1. Proposed ensemble. All four heads use the same 64-D CNN embedding; three combiners are compared with the CNN head alone.

C. Combination Strategies

Majority vote flags a clip as HS when at least half of the members do (hard decisions at $p \geq 0.5$; with four members a 2-2 tie is resolved as HS). Weighted vote averages member probabilities with non-negative weights summing to one (each member's weight is its balanced accuracy on the validation split, normalised) and thresholds the result:

$$s(x) = \sum_{m=1}^M w_m p_m(x), \quad w_m \geq 0, \quad \sum_m w_m = 1 \quad \tag{3}$$

with a clip flagged as HS when $s(x) \geq \tau = 0.5$. Stacking [15] instead trains a meta-learner g (logistic regression with balanced class weights) on the member outputs,

$$\hat{y} = g(p_1(x), \dots, p_M(x)) \quad \tag{4}$$

so that it can learn which member to trust. The vote weights and g are fitted on the validation split only (a stratified 15% of the training partition), never on the test partition. That split is also used to early-stop the CNN, and the classical heads are trained on training-split embeddings from a CNN that has already seen the training labels, so they see cleaner embeddings than validation or test clips.

V. EXPERIMENTAL SETUP

Protocol. Each of the five benchmarks is trained and tested separately on its official training and test partitions. Clips are converted to grayscale, resized to 48×48 and scaled to [0, 1]; a stratified 15% of each training partition is held out as validation data. All models, including the baseline, use the same preprocessing and the same partitions; the test partition is used only for the final scores. Every model is scored with the metrics of Section III-B, and averages over benchmarks are unweighted (macro) means. Table II documents the configuration.

TABLE II: EXPERIMENTAL CONFIGURATION

Component	Setting
---	---
Data	Official ICCAD-12 train/test partitions; five benchmarks handled separately (no merging).
CNN input	Grayscale, bilinear resize to 48×48, scaled to [0, 1].
SVM/RF/ANN input	64-D CNN embedding (standardised for SVM and ANN).
Validation	15% stratified hold-out of the training partition (seed 42); used for CNN early stopping, vote weights and the meta-learner.
Baseline CNN	Two basic blocks (16, 32 filters) + 64-unit embedding, ·323 k params.; Nadam (10^{-3}), BCE with hotspot class weight, batch 32, 15 epochs, early stopping on val. AUC (patience 5).
SVM	RBF kernel, Platt probabilities, balanced class weights.
Random Forest	300 trees, balanced class weights.
ANN	MLP (128, 64), 500 iterations, internal early stopping, no class weights.

Majority vote	Hard vote over four members; 2·2 tie $\rightarrow$ HS.
Weighted vote	Weights = validation BA of each member (normalised); $\tau = 0.5$.
Stacking	Logistic regression, balanced class weights, fitted on validation-split probabilities.
Seeds / runs	Seed 42; one run per benchmark.
Software / HW	Python ≥ 3.10 , TensorFlow ≥ 2.10 , scikit-learn ≥ 1.0 ; Kaggle GPU.

VI. RESULTS AND ANALYSIS

A. Baseline versus Ensembles

Table III and Fig. 2 give BA for every model and benchmark. Stacking attains the highest average BA, 92.97%, which is +0.88 pp above the baseline CNN (92.09%) and +4.1 pp above the best classical member (SVM, 88.86%). Majority and weighted voting fall 2.2 and 2.1 pp below the baseline. H1 is therefore supported for stacking only, and only by a small margin. Stacking beats the baseline on ICCAD-1 to -3 (+3.28, +1.87, +1.39 pp) but is lower on ICCAD-4 (-1.54 pp) and ICCAD-5 (-0.62 pp), the two most imbalanced benchmarks, where the baseline CNN is strongest.

TABLE III: BALANCED ACCURACY (%) PER BENCHMARK

Bench.	CNN	SVM	RF	ANN	Maj.	Wtd.	Stack
---	---	---	---	---	---	---	---
B1	82.83	86.60	85.54	83.59	83.77	86.18	86.11
B2	90.79	87.89	87.21	86.09	90.78	88.90	92.66
B3	95.71	97.20	97.05	96.95	97.11	97.18	97.10
B4	93.73	86.34	82.57	79.22	88.06	85.58	92.19
B5	97.42	86.26	88.71	50.00	89.86	92.29	96.80
Avg.	92.09	88.86	88.22	79.17	89.92	90.03	92.97

CNN = baseline. RF = Random Forest; Maj./Wtd. = majority/weighted vote. Best value per row in bold.

Table IV shows the other metrics averaged over benchmarks. The CNN is a high-recall, lower-specificity detector ($R = 92.3\%$, $S = 91.9\%$), whereas SVM and RF are the opposite ($R \approx 84\%$, $S \approx 93.6\%$). Voting ensembles inherit the classical members' conservatism: recall drops to 87.4% (majority) and 86.5% (weighted) while precision and F1 improve. Stacking is the only ensemble that keeps the CNN's recall (92.7%) and also raises specificity to 93.2%, lifting F1 from 46.7% to 48.6%. Low absolute precision ($< 43\%$ for all models) follows from prevalence, Eq. (2).

TABLE IV: AVERAGE METRICS OVER THE FIVE BENCHMARKS (%)

Model	BA	Prec.	Recall	Spec.	F1
---	---	---	---	---	---
CNN (baseline)	92.09	37.72	92.28	91.91	46.75
SVM	88.86	39.63	84.05	93.67	49.94
Random Forest	88.22	42.58	82.86	93.57	51.66
ANN	79.17	37.48	65.76	92.57	43.36
Majority vote	89.92	39.12	87.35	92.48	49.93
Weighted vote	90.03	41.22	86.50	93.55	51.36
Stacking	92.97	37.26	92.69	93.25	48.59

ANN precision on ICCAD-5 is 0/0 (no positive predictions) and is counted as 0. Best value per column in bold.

B. Per-Benchmark Metrics and Confusion Matrices

Tables V and VI and Fig. 3 give the full per-benchmark results and confusion matrices. The differences between baseline and stacking are of different kinds on different benchmarks. On ICCAD-1 both detect all 226 hotspots and stacking removes 307 of 1,607 false alarms (specificity 65.7% $\rightarrow$ 72.2%), yet precision stays at 12·15% for every model, so ICCAD-1 is the hardest benchmark for false alarms. On ICCAD-2 stacking finds 19 more of the 498 hotspots (recall 81.9% $\rightarrow$ 85.7%) for 29 more false alarms. On ICCAD-3 it cuts missed hotspots from 123 to 49

(.60%) but adds 608 false alarms. On ICCAD-4 it misses 6 more hotspots but raises 99 fewer false alarms; on ICCAD-5 it misses one more hotspot but raises 233 fewer false alarms. Pooled over all test sets (Table VI), stacking misses 148 hotspots against 234 for the baseline (.36.8%) with essentially the same number of false alarms (3,500 vs 3,502), which supports H2 for stacking. Weighted voting is also better than the baseline in pooled counts (214 misses, 3,113 false alarms) yet worse in macro-average BA, because pooled counts are dominated by the large ICCAD-2/3 test sets whereas the macro average gives ICCAD-4/5 equal weight.

TABLE V: PER-BENCHMARK METRICS (%) OF BASELINE AND ENSEMBLES

Bench.	Model	BA	Prec.	Recall	Spec.	F1
----	----	----	----	----	----	----
B1	CNN	82.83	12.33	100.00	65.66	21.95
Maj.		83.77	12.95	100.00	67.54	22.93
Wtd.		86.18	14.88	100.00	72.37	25.90
Stack		86.11	14.81	100.00	72.22	25.80
B2	CNN	90.79	73.78	81.93	99.65	77.64
Maj.		90.78	73.38	81.93	99.64	77.42
Wtd.		88.90	75.39	78.11	99.69	76.73
Stack		92.66	71.05	85.74	99.58	77.71
B3	CNN	95.71	67.10	93.20	98.22	78.03
Maj.		97.11	53.68	97.51	96.72	69.25
Wtd.		97.18	56.47	97.29	97.07	71.46
Stack		97.10	55.09	97.29	96.91	70.35
B4	CNN	93.73	28.34	88.70	98.76	42.95
Maj.		88.06	37.26	76.84	99.28	50.18
Wtd.		85.58	40.45	71.75	99.41	51.73
Stack		92.19	33.63	85.31	99.07	48.24
B5	CNN	97.42	7.05	97.56	97.27	13.16
Maj.		89.86	18.33	80.49	99.24	29.86
Wtd.		92.29	18.92	85.37	99.22	30.97
Stack		96.80	11.71	95.12	98.48	20.86
Avg.	CNN	92.09	37.72	92.28	91.91	46.75
Maj.		89.92	39.12	87.35	92.48	49.93
Wtd.		90.03	41.22	86.50	93.55	51.36
Stack		92.97	37.26	92.69	93.25	48.59

TABLE VI: CONFUSION-MATRIX COUNTS (HS = POSITIVE CLASS)

Bench.	Model	TP	FP	FN	TN
----	----	----	----	----	----
B1	CNN	226	1,607	0	3,072
Maj.		226	1,519	0	3,160
Wtd.		226	1,293	0	3,386
Stack		226	1,300	0	3,379
B2	CNN	408	145	90	41,153
Maj.		408	148	90	41,150
Wtd.		389	127	109	41,171
Stack		427	174	71	41,124
B3	CNN	1,685	826	123	45,507
Maj.		1,763	1,521	45	44,812
Wtd.		1,759	1,356	49	44,977
Stack		1,759	1,434	49	44,899
B4	CNN	157	397	20	31,493
Maj.		136	229	41	31,661
Wtd.		127	187	50	31,703

	Stack		151		298		26		31,592			
	B5		CNN		40		527		1		18,800	
	Maj.		33		147		8		19,180			
	Wtd.		35		150		6		19,177			
	Stack		39		294		2		19,033			
	Pooled		CNN		2,516		3,502		234		140,025	
	Maj.		2,566		3,564		184		139,963			
	Wtd.		2,536		3,113		214		140,414			
	Stack		2,602		3,500		148		140,027			

C. Which Errors Does Combination Reduce?

Fig. 4 places every model at its average operating point. The CNN sits at high recall but a false-positive rate of 8.09%; SVM and RF sit at lower recall but a lower false-positive rate; the ANN has by far the lowest recall (65.8%) because it detects no hotspot at all on ICCAD-5 and only 58.8% on ICCAD-4. Fixed-rule ensembles land between the members, in part because they threshold the raw probabilities of members that are not calibrated to a common scale. Majority voting, for example, loses hotspots that the CNN alone catches: on ICCAD-4/5 its recall is 76.8%/80.5% against the CNN's 88.7%/97.6%, because three weaker members outvote the one strong member. Stacking is the only combiner to move up and to the left of the baseline (higher recall and a lower false-positive rate, 6.75%), which is consistent with a learned meta-learner re-scaling and re-weighting the members. We did not test whether this reflects complementary errors or merely re-scaling of the CNN's own probability; a CNN re-calibrated on the validation split is a control we have not run. Per benchmark, stacking mainly removes **misses** on ICCAD-2 and -3 and **false alarms** on ICCAD-1, -4 and -5. Errors shared by all members are not removed: on ICCAD-1 every model has recall $\geq 98.7\%$ but specificity of only 65.7-73.2%, so the false alarms there are largely common-mode. Plausible but untested reasons are ICCAD-1's small training set (439 clips) and the shared embedding.

D. Ablation: Leave-One-Out Members

To quantify each member's contribution, one member at a time is removed from the weighted-vote ensemble (full ensemble: 90.03% average BA). Table VII and Fig. 5 show the change. Removing the CNN's own output (the classical heads still use its embedding) causes the largest drop (-2.74 pp on average, -10.83 pp on ICCAD-5), so the CNN head is the most valuable member, and its contribution is concentrated on the imbalanced ICCAD-4/5. Removing the SVM costs 0.51 pp. Removing the RF (+0.41 pp) or the ANN (+0.90 pp) improves the ensemble; the ANN's gain comes almost entirely from ICCAD-4 and -5, where it is the weakest member. The three classical heads alone (CNN head removed) reach 87.29%, which is below the best of them (SVM, 88.86%): combining models is not automatically beneficial, and the CNN's own head is what makes the ensemble competitive. The ablation was run on the weighted-vote ensemble only; the stacking ensemble was not ablated.

TABLE VII: LEAVE-ONE-OUT ABLATION, WEIGHTED-VOTE ENSEMBLE: BA (%)

Bench.	Full	· CNN	· SVM	· RF	· ANN
· ---	· ---	· ---	· ---	· ---	· ---
B1	86.18	86.27	85.35	85.31	86.28
B2	88.90	87.90	88.60	89.09	88.69
B3	97.18	97.15	97.18	97.24	97.24
B4	85.58	83.65	84.25	85.84	87.80
B5	92.29	81.46	92.20	94.70	94.66
Avg.	**90.03**	**87.29**	**89.52**	**90.44**	**90.93**

· CNN· removes the CNN's own output only; the SVM, RF and ANN heads still use its embedding.

E. Comparison with the Reference Work and Validity

The reference work reports 95.3% average BA for the CNN and 97.9% on ICCAD-2 [10], against 92.09% and 90.79% here. These numbers are not directly comparable: [10] reports *·validation·* BA without stating the partition, whereas ours are computed on test partitions whose sizes match the official ones (Table I). Both agree that the CNN is a strong single model. The baseline CNN also differs from the reference network (about 323 k vs 12,873

parameters, Section IV-A), which is a further reason not to compare absolute numbers. We did not measure inference time: our CNN is about 25 times larger than the one in [10], so the 6.3 ms per clip reported there does not carry over, and the ensemble additionally runs an SVM, a Random Forest, an ANN and a meta-learner on the embedding.

****Statistical uncertainty.**** Each benchmark was evaluated once. Treating recall and specificity as binomial proportions, the standard error of a single benchmark's BA is $0.2 \cdot 1.7$ pp for stacking (largest on ICCAD-5, which has 41 hotspots) and the standard error of the five-benchmark average is about $0.4 \cdot 0.5$ pp for both the CNN and stacking. The $+0.88$ pp average gain, and the ICCAD-4/5 deficits, are therefore comparable to the sampling noise even before training randomness, so they should be read as trends rather than significant differences. Only the pooled reduction in missed hotspots and the ICCAD-1 and ICCAD-3 gains appear to exceed this range clearly.

VII. DISCUSSION, LIMITATIONS AND FUTURE WORK

****Meaning of the results.**** Heterogeneity helps only when the combiner can exploit it. A fixed vote treats the CNN head and three weaker classical heads as equals and lets the majority overrule the strongest, whereas stacking preserves the CNN's sensitivity and borrows the classical heads' lower false-alarm rate. Because all heads share the CNN's embedding, the benefit comes from differences in decision rule and probability scaling, not from independent views of the layout. For CAD use, where a missed hotspot is far costlier than a false alarm, the 37% pooled reduction in misses is the most relevant outcome; the price is three extra classifiers and a meta-learner on top of the CNN pass (cost not measured here).

****Limitations.**** (i) Single run per benchmark (seed 42), with no confidence intervals or paired significance tests; GPU training is not bit-wise reproducible. (ii) The members share the CNN's embedding, so their errors are correlated, and we did not test whether the stacking gain reflects complementary errors or simply re-scaling of the CNN's probability on the validation split. (iii) The validation split is small for the imbalanced benchmarks (about 4, 14 and 15 hotspots for ICCAD-5, -4 and -1), yet it is used for CNN early stopping and to fit the vote weights and the meta-learner. (iv) The best ensemble was identified from test-set scores, so all three variants are reported. (v) The ablation covers weighted voting only and removes the CNN's own output, not its embedding. (vi) Ensemble latency was not measured. (vii) The ANN is an MLP without class weighting and predicts no hotspot on ICCAD-5, where only about 22 hotspots remain for training, so its weak result reflects the setup as much as the model class. (viii) Only within-benchmark performance was tested, and the baseline is a larger variant of the reference CNN, so we make no comparison with state-of-the-art detectors such as [7] or with the absolute numbers of [10]. (ix) The ICCAD-1 test-set count differs from [10] (Table I).

****Future work.**** Promising directions are: repeated runs with paired tests (e.g., bootstrap or McNemar); a re-calibrated or re-thresholded CNN as a stronger single-model control; members with genuinely different inputs (e.g., independently trained CNNs or classical models on hand-crafted layout features); out-of-fold stacking; cost-sensitive stacking or threshold tuning to target a recall level; pruning weak members such as the ANN; a two-stage cascade in which the CNN screens and the ensemble re-examines only uncertain clips; and measured latency-accuracy trade-offs.

VIII. CONCLUSION

We compared a CNN baseline with majority-vote, weighted-vote and stacking ensembles of the CNN head and SVM, Random Forest and ANN classifiers built on its embedding, on all five ICCAD-12 benchmarks. Stacking gave the best average balanced accuracy (92.97% vs 92.09%) and cut pooled missed hotspots by 37% at equal false alarms, while simple voting did not beat the CNN. The CNN's own output is the most valuable member, the ANN is a liability, and the benefit is benchmark-dependent and modest relative to single-run uncertainty. Because the members share one embedding, the gain may partly reflect probability re-scaling rather than complementary errors. A learned combiner, rather than a fixed vote, is thus the better way to combine these heads for hotspot detection; confirming the gain requires repeated runs, a re-calibrated single-CNN control and latency measurements.

AUTHOR CONTRIBUTIONS

TABLE VIII: WORK ALLOCATION

Task	Member(s)
---	---
Literature survey and problem definition	A. K. Kudipudi
ICCAD-12 data and class-imbalance analysis	A. K. Kudipudi
Baseline CNN implementation and evaluation	Tejasvini R
SVM, Random Forest and ANN members	Tejasvini R
Voting and stacking ensembles	S. P. Gupta
Ablation, error analysis, slides and documentation	S. P. Gupta

CODE AVAILABILITY

Code, notebooks, results (CSV) and the README with execution instructions: [Github Link](<https://github.com/Shubham-G04/Heterogeneous-ML-DL-Ensemble-for-Reliable-Lithography-Hotspot-Detection>). The baseline CNN is a variant of the architecture of [13], [10] (Section IV-A).

ACKNOWLEDGMENT

The authors thank Dr. G. Lakshmi Priya for the problem statement and guidance.

REFERENCES

1. H. Yang, Y. Lin, B. Yu, and E. F. Y. Young, "Lithography hotspot detection: From shallow to deep learning," in *Proc. 30th IEEE Int. Syst.-on-Chip Conf. (SOCC)**, Munich, Germany, 2017, pp. 233-238.
2. W.-Y. Wen, J.-C. Li, S.-Y. Lin, J.-Y. Chen, and S.-C. Chang, "A fuzzy-matching model with grid reduction for lithography hotspot detection," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 33, no. 11, pp. 1671-1680, Nov. 2014.
3. Y.-T. Yu, G.-H. Lin, I. H.-R. Jiang, and C. Chiang, "Machine-learning-based hotspot detection using topological classification and critical feature extraction," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 34, no. 3, pp. 460-470, Mar. 2015.
4. T. Matsunawa, J.-R. Gao, B. Yu, and D. Z. Pan, "A new lithography hotspot detection framework based on AdaBoost classifier and simplified feature extraction," in *Proc. SPIE**, vol. 9427, 2015, Art. no. 94270S.
5. H. Zhang, B. Yu, and E. F. Y. Young, "Enabling online learning in lithography hotspot detection with information-theoretic feature optimization," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)**, 2016, Art. no. 47.
6. H. Yang, L. Luo, J. Su, C. Lin, and B. Yu, "Imbalance aware lithography hotspot detection: A deep learning approach," *J. Micro/Nanolithogr. MEMS MOEMS**, vol. 16, no. 3, Art. no. 033504, 2017.
7. H. Yang, J. Su, Y. Zou, Y. Ma, B. Yu, and E. F. Y. Young, "Layout hotspot detection with feature tensor generation and deep biased learning," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 38, no. 6, pp. 1175-1187, Jun. 2019.
8. J. A. Torres, "ICCAD-2012 CAD contest in fuzzy pattern matching for physical verification and benchmark suite," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)**, 2012, pp. 349-350.
9. K. H. Brodersen, C. S. Ong, K. E. Stephan, and J. M. Buhmann, "The balanced accuracy and its posterior distribution," in *Proc. 20th Int. Conf. Pattern Recognit. (ICPR)**, 2010, pp. 3121-3124.
10. A. Verma, K. A. Rao, and D. S. Hegde, "Lithography hotspot detection using deep learning," EE769 course project report, IIT Bombay. [Online]. Available: <https://github.com/Intelectron6/Lithography-Hotspot-Detection>

(accessed Sep. 20, 2026).

11. K. Simonyan and A. Zisserman, ·Very deep convolutional networks for large-scale image recognition,· in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.
12. L. Liao, S. Li, Y. Che, W. Shi, and X. Wang, ·Lithography hotspot detection method based on transfer learning using pre-trained deep convolutional neural network,· *Appl. Sci.*, vol. 12, no. 4, Art. no. 2192, 2022.
13. V. Borisov and J. Scheible, ·Lithography hotspots detection using deep learning,· in *Proc. 15th Int. Conf. Synthesis, Modeling, Anal. Simul. Methods Appl. Circuit Design (SMACD)*, Prague, Czech Republic, 2018, pp. 145-148.
14. T. G. Dietterich, ·Ensemble methods in machine learning,· in *Multiple Classifier Systems (LNCS 1857)*. Berlin, Germany: Springer, 2000, pp. 1-15.
15. D. H. Wolpert, ·Stacked generalization,· *Neural Netw.*, vol. 5, no. 2, pp. 241-259, 1992.
16. O. Sagi and L. Rokach, ·Ensemble learning: A survey,· *WIREs Data Mining Knowl. Discovery*, vol. 8, no. 4, Art. no. e1249, 2018.